

Contents

ENGY 680 Tutorial: Setting Up Your Python + GitHub Environment	1
Attribution	2
Table of Contents	2
0. Before you start	2
1. Install conda (Miniconda) or Anaconda	2
🍏 Mac — Option A: Miniconda installation (recommended)	3
🍏 Mac / 🖥️ PC — Option B: Graphical Anaconda installer with Navigator application	4
🖥️ PC note	5
Verify the install	5
2. Install GitHub Desktop and sign in	5
3. Clone your private course repo	6
4. Create and activate the conda environment	8
Option A: Command line (Mac or PC)	8
Option B: Anaconda Navigator (graphical, no terminal) . .	8
5. Launch Jupyter Lab	10
Command line	10
Anaconda Navigator	11
6. Run the demo/homework notebook	13
7. Commit and push your changes	15
8. Troubleshooting	19

ENGY 680 Tutorial: Setting Up Your Python + GitHub Environment

This tutorial covers the steps needed to get your computer ready for Urban Energy and Climate problem sets: installing Python (via conda / Anaconda), installing GitHub Desktop, cloning your personal course repo, running a Jupyter notebook, and committing + pushing your work back to GitHub.

Who this is for: students with no prior command-line, Python, or Git experience. Every step works on both Mac and PC (Windows) — where the steps diverge, look for the 🍏 Mac / 🖥️ PC labels.

We will practice these steps in class together. Before the first class, just pay attention to Step 0, though if you have time, you may want to get a headstart on installations to save WiFi bandwidth in class.

Note that these steps are primarily tested on a Mac. Behavior on Windows is somewhat dependent on software versions and thus unpredictable. If you encounter persistent problems, reach out to the instructor, TA, or other students to see if there may be a platform-specific challenge or error in these instructions.

Attribution

Claude Sonnet 5 (with reasoning) was used to draft this tutorial from a description of the context, links to the README and homework instructions, and an outline of the 7 major steps: see conversation via Kagi.

Table of Contents

0. Before you start
 1. Install conda (Miniconda)
 2. Install GitHub Desktop and sign in
 3. Clone your private course repo
 4. Create and activate the conda environment
 5. Launch Jupyter Lab
 6. Run the demo/homework notebook
 7. Commit and push your changes
 8. *Troubleshooting* <!--
 9. Screenshot checklist -->
-

0. Before you start

- You have a github.com account.
 - You’ve emailed the instructor your GitHub username (or accepted the emailed repo invite), so your private repo [YOUR_USERNAME] /engy680 exists and you have access.
 - You have ~2 GB of free disk space (if installing Miniconda) or ~10 GB (if installing the full Anaconda with graphical Navigator) and a decent internet connection.
-

1. Install conda (Miniconda) or Anaconda

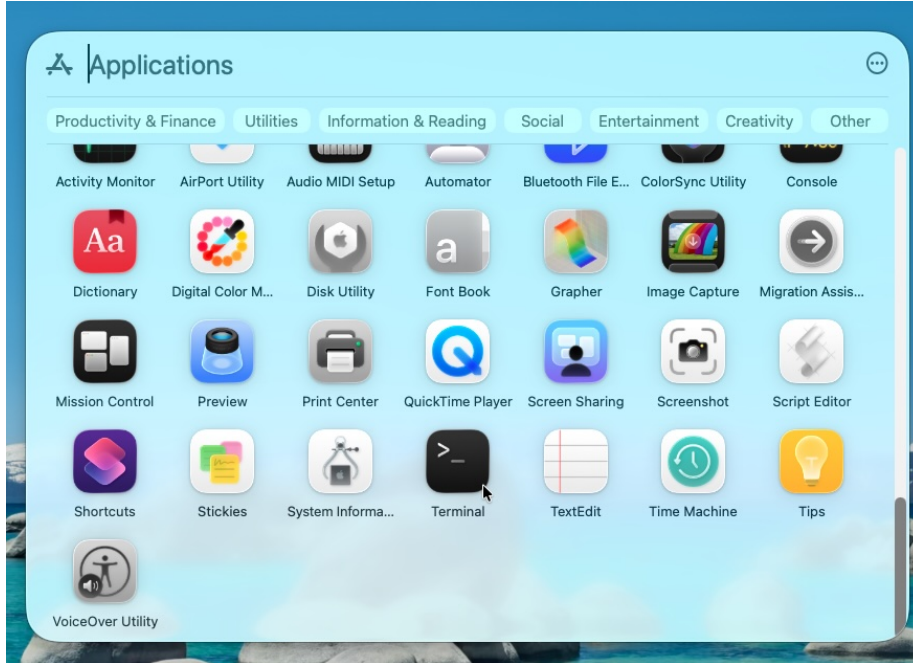
Conda is a tool for managing isolated Python installations (“environments”) so that every student in the class runs the *same* package versions, regardless of what’s already on your computer. Miniconda (a minimal installer) is preferred to the full Anaconda Distribution to minimize disk space and the chance of conflicting environment dependencies (libraries explicitly or implicitly required by our notebooks invoked using “import” commands), since we’ll build our own environment from `environment.yml` anyway. Miniconda is designed to run entirely from the command line (Terminal).

The full Anaconda distribution works fine too if you already have it, and it is recommended on Windows.

🍏 Mac — Option A: Miniconda installation (recommended)

You can install via Terminal or graphically.

Option A1. Install Miniconda from Terminal using brew: Open the Terminal app (Spotlight search or Applications window → “Terminal”),



then run:

```
brew install --cask miniconda
```

If you don't have Homebrew yet, install it first from brew.sh, or install graphically.

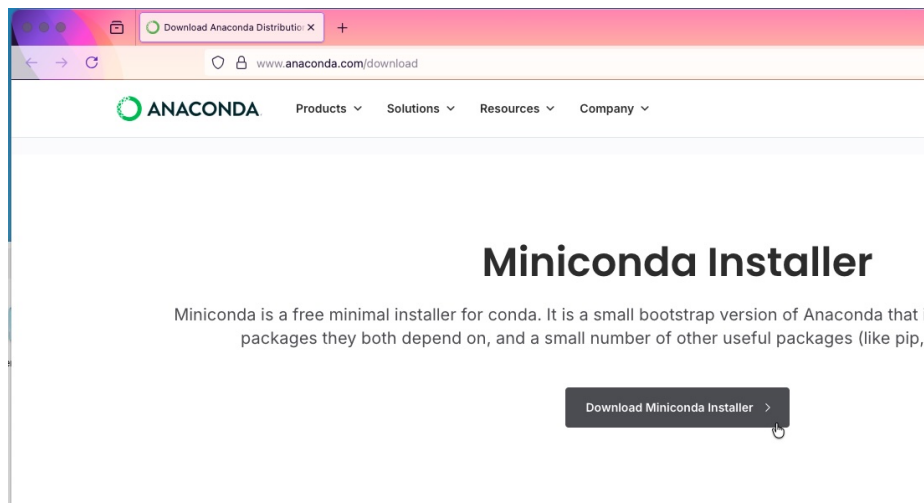
Option A2. Graphical installation for Miniconda: Go to anaconda.com/download and download the Miniconda installer. Run the installer and follow the prompts (accept defaults unless you have a reason not to; on the “Install for” screen choose Just Me, not All Users).

```
zsubin -- -bash -- 80x24

Last login: Sun Aug 23 12:35:48 on ttys001

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) ML-ITS-260127:~ zsubin$ brew install --cask miniconda
```

Figure 1: Step 1: install conda



🍏 Mac / 💻 PC — Option B: Graphical Anaconda installer with Navigator application

Go to anaconda.com/download and download the Anaconda installer. Run the installer and follow the prompts (accept defaults unless you have a reason not to; on the “Install for” screen choose Just Me, not All Users).

📌 PC note

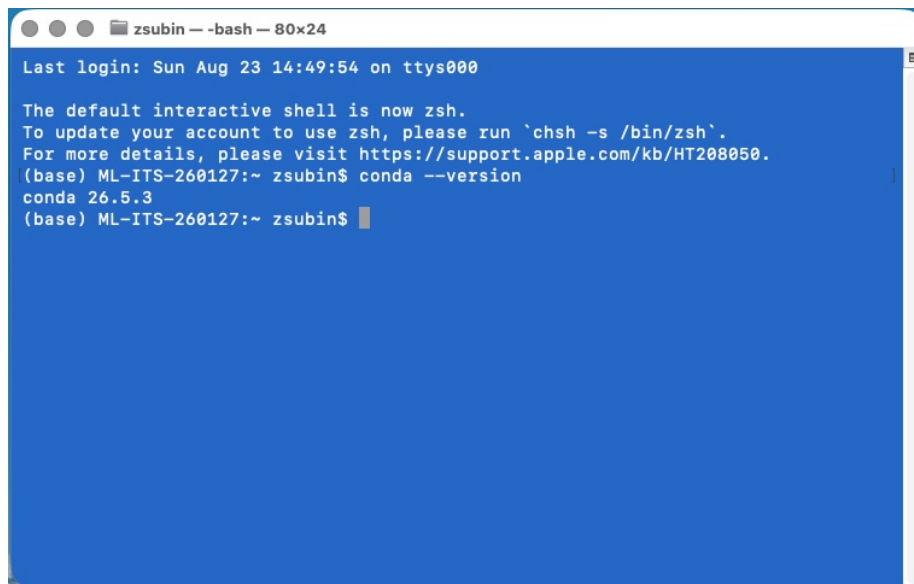
On Windows, the installer adds an Anaconda Prompt to your Start Menu — use this instead of the regular Command Prompt / PowerShell for all conda or jupyter commands below or just use Navigator in this case (unless you checked the box during install to add conda to your system PATH).

Verify the install

Open Terminal (Mac) or Anaconda Prompt (PC) and run:

```
conda --version
```

You should see something like conda 24.x.x. If you get “command not found,” see Troubleshooting.

A screenshot of a terminal window with a blue background. The window title is 'zsubin -- bash -- 80x24'. The text inside shows a login session on 'Sun Aug 23 14:49:54 on ttys000'. It states 'The default interactive shell is now zsh.' and provides instructions to update the account to use zsh by running 'chsh -s /bin/zsh'. Below this, the user runs 'conda --version' and the output is 'conda 26.5.3'. The prompt '(base) ML-ITS-260127:~ zsubin\$' is visible at the end of the line.

```
zsubin -- bash -- 80x24
Last login: Sun Aug 23 14:49:54 on ttys000
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
(base) ML-ITS-260127:~ zsubin$ conda --version
conda 26.5.3
(base) ML-ITS-260127:~ zsubin$
```

Figure 2: Step 1: Terminal showing successful conda --version output

2. Install GitHub Desktop and sign in

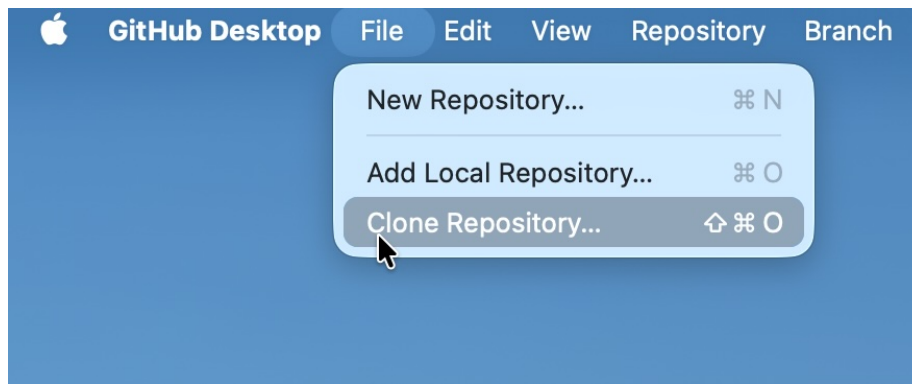
We’ll use GitHub Desktop — a graphical app — instead of raw git commands, so you never have to memorize git syntax, and to simplify logging into github (you can no longer log in to access private repos from a terminal simply with a username and password; you have to use an ssh-key or OAuth procedure which GitHub Desktop takes care of for you).

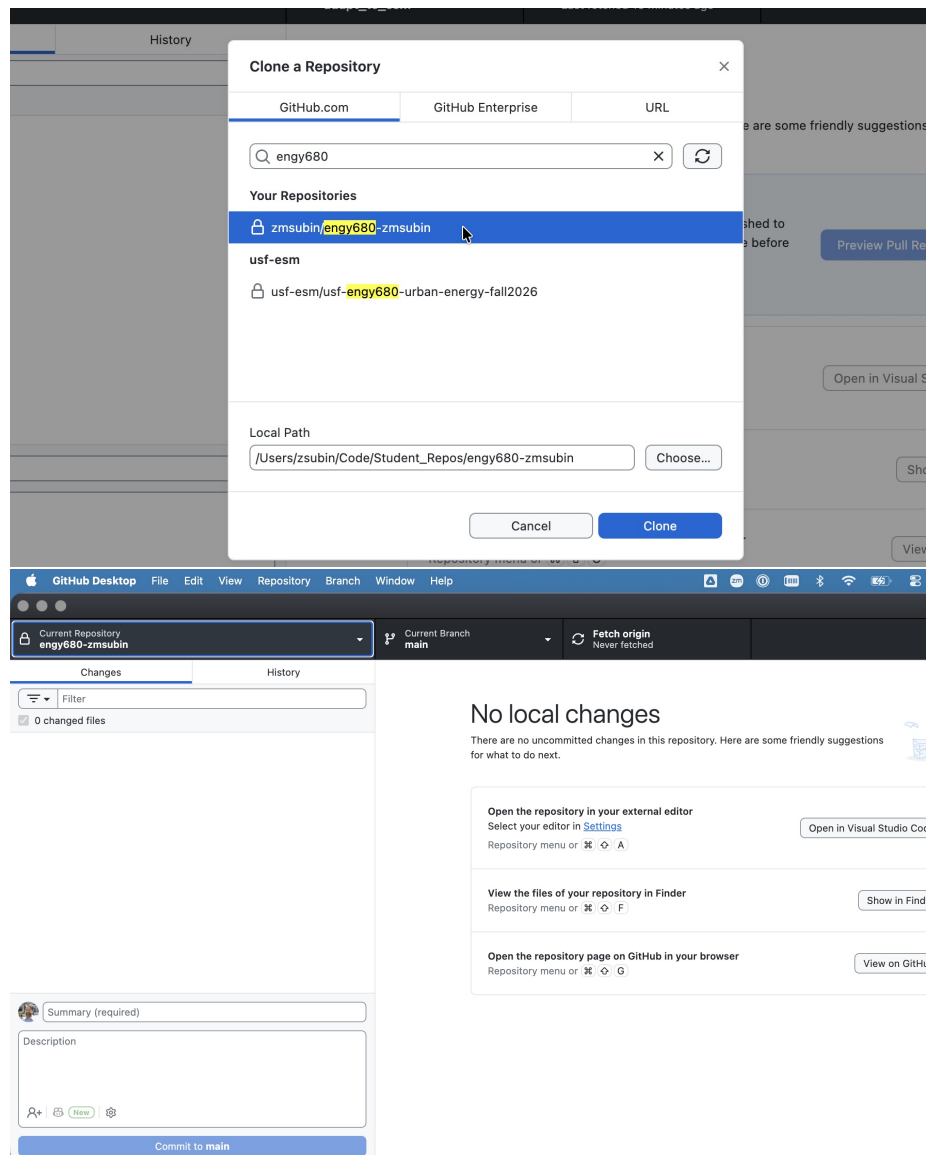
1. Download it from desktop.github.com. (You can alternatively install this via brew using the “github” cask.)
 2. Install it:
 - 🍏 Mac: open the .zip, drag GitHub Desktop into Applications.
 - 💻 PC: run the installer .exe; it installs and launches automatically.
 3. Open GitHub Desktop and click Sign in to GitHub.com. This opens your browser to authorize the app — log in with your GitHub account and click Authorize GitHub Desktop, then return to the app.
 4. If prompted, confirm your name/email for git commits (your GitHub account name/email is fine).
-

3. Clone your private course repo

Each student has a private repo named [INSTRUCTOR_USERNAME] / engy680- [YOUR_USERNAME] (a copy of the shared course repo). 🚫 DO NOT try to do homework in the shared/instructor repo — only your private copy. (The GitHub permissions should be configured to prevent this, but just in case they are not, this will keep your homework private.)

1. In GitHub Desktop, go to File → Clone Repository...
2. Click the GitHub.com tab and find [INSTRUCTOR_USERNAME] / engy680- [YOUR_USERNAME] in the list (or paste the URL under the URL tab if it doesn't show up).
3. Choose a Local Path — somewhere easy to find, e.g. Documents / engy680.
4. Click Clone and wait for it to finish.





If your repo isn't listed: your invite may not be accepted yet — check your email (including spam) for a GitHub invite, or confirm with the instructor that your GitHub username was received.

4. Create and activate the conda environment

Your repo includes an `environment.yml` file that lists every Python package the class needs (pandas, jupyterlab, matplotlib, etc.), so everyone has identical versions.

Option A: Command line (Mac or PC)

Open Terminal / Anaconda Prompt, navigate into your cloned repo, and create the environment:

```
cd path/to/engy680
conda env create -f environment.yml -n engy680
```

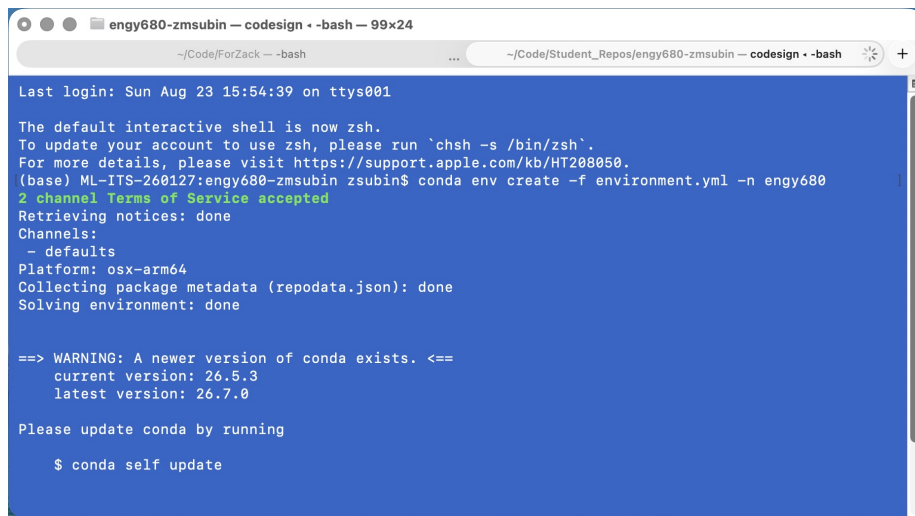


Figure 3: Step 4: installing env

This can take a few minutes (“solving environment...”). When it finishes, activate it:

```
conda activate engy680
```

Your terminal prompt should now show `(engy680)` at the start of the line.

Verify:

```
conda env list
```

You should see `engy680` in the list with an asterisk `*` next to it if active.

Option B: Anaconda Navigator (graphical, no terminal)

1. Open Anaconda Navigator.

```
engy680-zmsubin -- -bash -- 99x24
~/Code/ForZack -- -bash
~/Code/Student_Repos/engy680-zmsubin -- -bash

# To deactivate an active environment, use
#
# $ conda deactivate

WARNING conda.pypi.main:notify_externally_managed_future(156):
Did you know? You can install many PyPI packages with conda
using the conda-pypi beta. Get started:
https://docs.conda.io/projects/conda/en/stable/new-features.html

(base) ML-ITS-260127:engy680-zmsubin zsubin$ conda activate engy680
(engy680) ML-ITS-260127:engy680-zmsubin zsubin$ conda env list

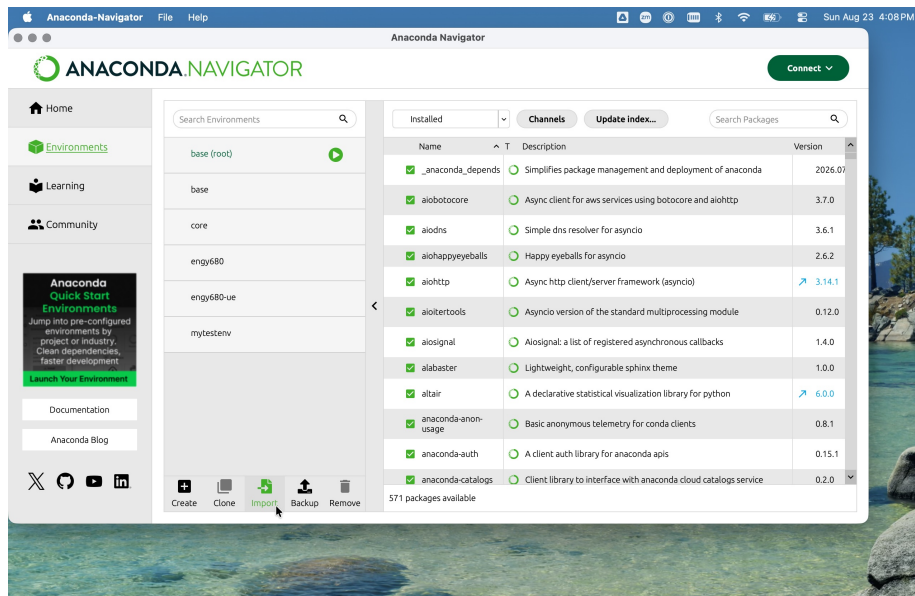
# conda environments:
#
# * -> active
# + -> frozen

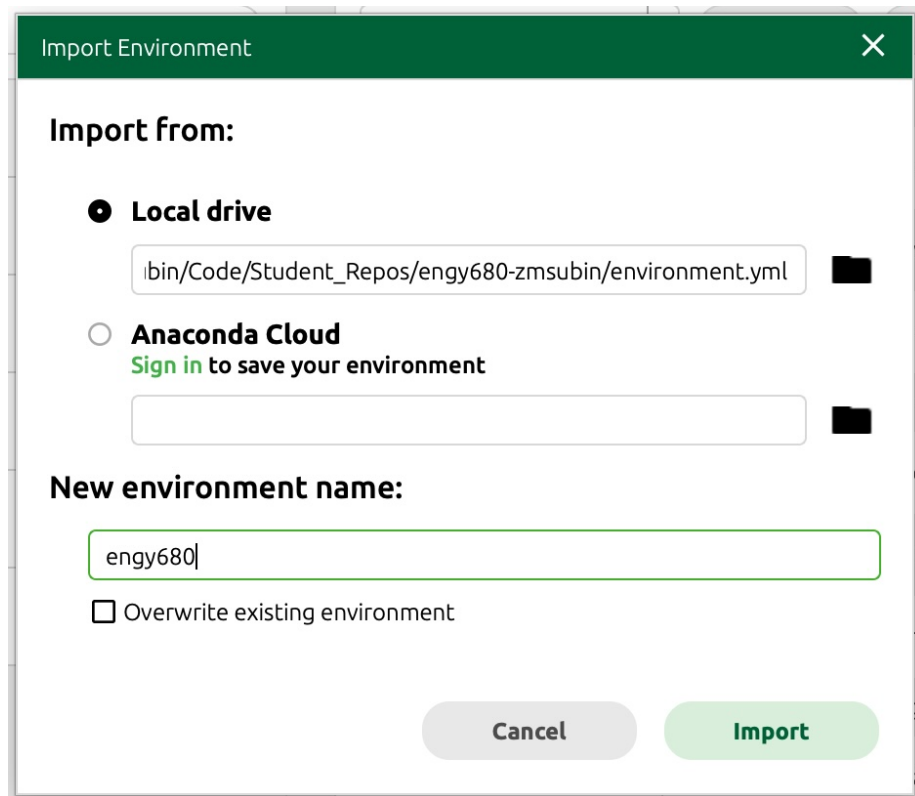
base                /Users/zsubin/Local_Apps/Anaconda3
core                /opt/homebrew/Caskroom/miniconda/base
engy680             * /opt/homebrew/Caskroom/miniconda/base/envs/core
engy680-ue          /opt/homebrew/Caskroom/miniconda/base/envs/engy680
engy680-ue          /opt/homebrew/Caskroom/miniconda/base/envs/engy680-ue
mytestenv           /opt/homebrew/Caskroom/miniconda/base/envs/mytestenv

(engy680) ML-ITS-260127:engy680-zmsubin zsubin$
```

Figure 4: Step 4: successful env

2. Click the Environments tab on the left.
3. Click Import at the bottom of the environments list.
4. Enter a name (e.g. engy680), browse to your repo's environment.yml under Specification File, and click Import.
5. Wait for the solver — this can take several minutes.



A dialog box titled "Import Environment" with a green header bar containing a close button (X). The main content area is white. Under the heading "Import from:", there are two radio button options. The first, "Local drive", is selected and has a text input field containing the path "ibin/Code/Student_Repos/engy680-zmsubin/environment.yml" and a small black square icon to its right. The second option, "Anaconda Cloud", is unselected and has the text "Sign in to save your environment" in green below it, followed by an empty text input field and another black square icon. Below these options is the heading "New environment name:" followed by a text input field containing "engy680". Underneath this field is a checkbox labeled "Overwrite existing environment". At the bottom right, there are two buttons: a grey "Cancel" button and a green "Import" button.

Import Environment

Import from:

☒ Local drive

ibin/Code/Student_Repos/engy680-zmsubin/environment.yml

☐ Anaconda Cloud

Sign in to save your environment

New environment name:

engy680

☐ Overwrite existing environment

Cancel Import

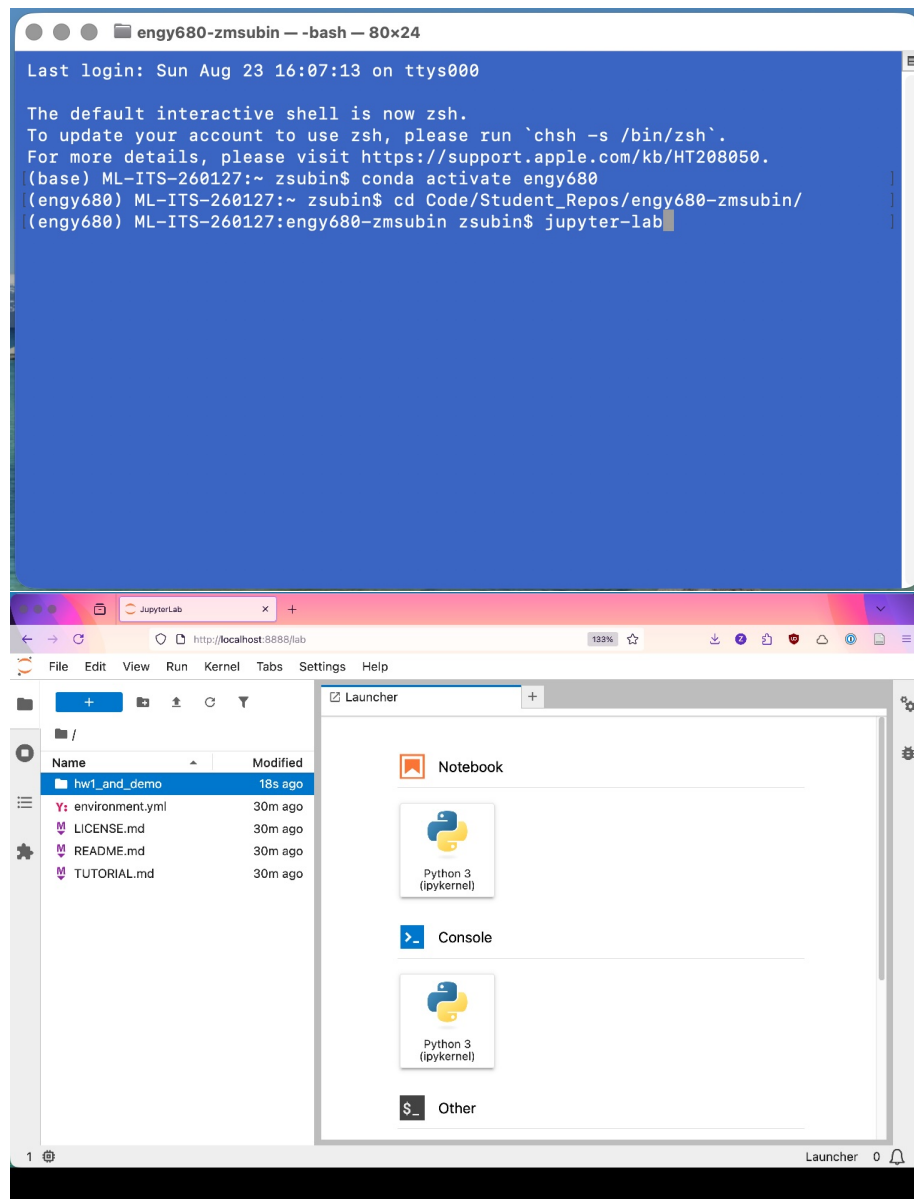
5. Launch Jupyter Lab

With the engy680 environment active, launch the notebook interface:

Command line

```
conda activate engy680    # if not already active
jupyter-lab
```

A browser tab should open automatically showing the Jupyter Lab file browser. (If not, copy the `http://localhost:8888/...` URL printed in the terminal into your browser.)

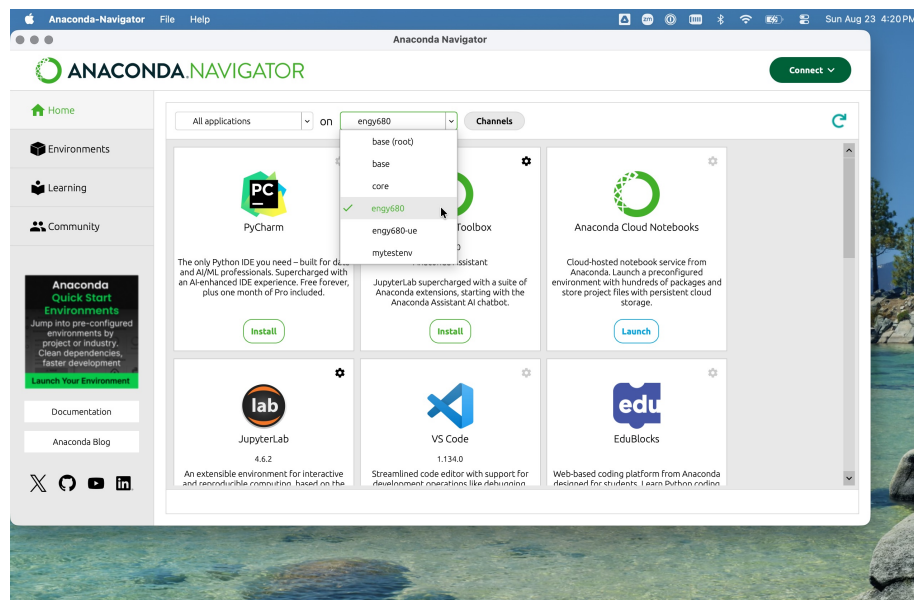


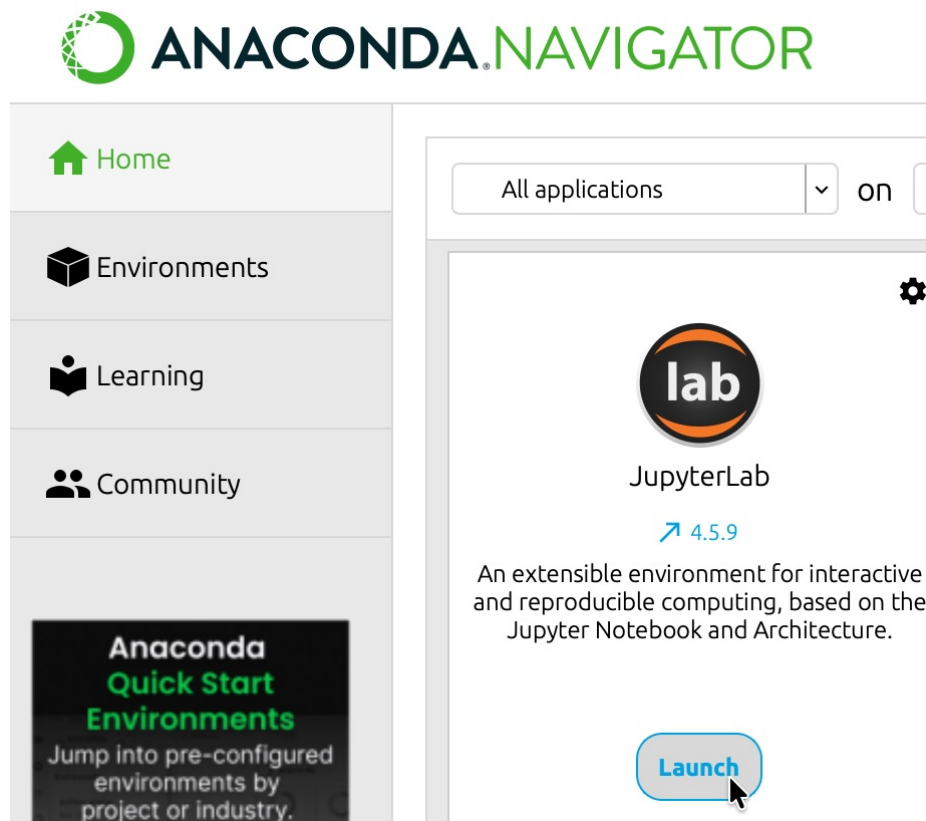
Anaconda Navigator

1. Go to the Home tab.
2. Make sure the environment dropdown at top reads engy680.
3. Click Launch under the JupyterLab tile.

If the JupyterLab tile shows an Install button instead of Launch after you select the engy680 environment, that usually means jupyterlab didn't make it into the environment during Step 4 (e.g., the import was interrupted). Click Install on the tile and wait for it to finish, or re-run the command-line install: `conda activate engy680 && conda install jupyterlab`.

🤔 This step may have problems on some configurations. Since we are using only very standard packages with common usages, you may be able to successfully run the homework using the Base environment that comes with the Navigator package. This may be less somewhat likely to work in subsequent homeworks when we make a map visualization using geopandas.



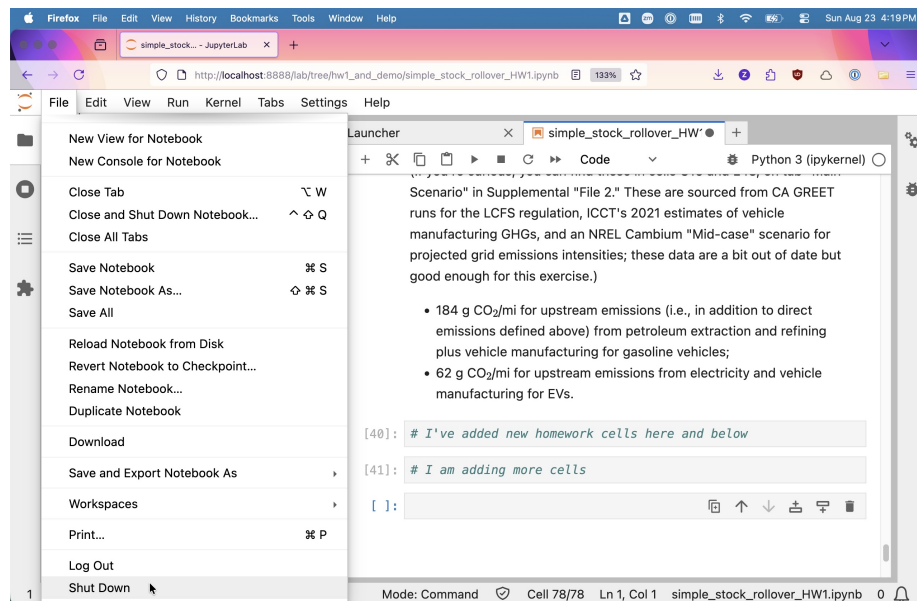


6. Run the demo/homework notebook

1. In the Jupyter Lab file browser (left sidebar), navigate to the homework folder and double-click the notebook file (e.g. `simple_stock_rollover_HW1.ipynb`).
2. Run the existing demo cells top to bottom: Run → Run All Cells (or click each cell and press Shift+Enter).
3. Scroll to the bottom, and add new cells for your homework answers:
 - Click the + icon in the toolbar, or press Esc then b (“insert cell Below”), to add a new cell.
 - Change a cell to Markdown (dropdown in toolbar) if you want to write text explanations, or leave it as Code.
4. Save frequently: Cmd/Ctrl + S, or File → Save Notebook. Jupyter autosaves periodically, but always do a manual save before stepping away.
5. When done, shut down cleanly:
 - File → Shut Down (stops the whole Jupyter Lab server), or
 - Close the browser tab and go back to the terminal, press Ctrl+C

twice to stop the server, or click Stop in Navigator.

The image displays two screenshots of a JupyterLab interface running in a Firefox browser. The top screenshot shows the initial state of the notebook 'simple_stock_rollover_HW1.ipynb'. The left sidebar shows the file explorer with the notebook file. The main area displays the notebook content, which includes a title 'Illustration of simple stock-rollover dynamics in light-duty vehicle sector', an introduction paragraph, and a code cell [1]: containing import statements for numpy, pandas, matplotlib.pyplot, and pathlib. Below the code cell, the text '1) Create our stock-rollover model.' is visible, followed by the start of a paragraph: 'First, we'll pick a time range. Let's create a year vector that ranges from'. The bottom screenshot shows the same notebook after execution. The code cell [1]: has been executed, and the output area now displays the text 'Scenario' in superscript. This is followed by a paragraph explaining that the data is sourced from CA GREET runs for the LCFS regulation, ICCT's 2021 estimates of vehicle manufacturing GHGs, and an NREL Cambium 'Mid-case' scenario for projected grid emissions intensities. Below this, there is a bulleted list of upstream emissions: 184 g CO₂/mi for upstream emissions (i.e., in addition to direct emissions defined above) from petroleum extraction and refining plus vehicle manufacturing for gasoline vehicles; and 62 g CO₂/mi for upstream emissions from electricity and vehicle manufacturing for EVs. The code cell [40]: is now empty, and a new code cell []: is visible at the bottom with the comment '# I am adding more cells'.



7. Commit and push your changes

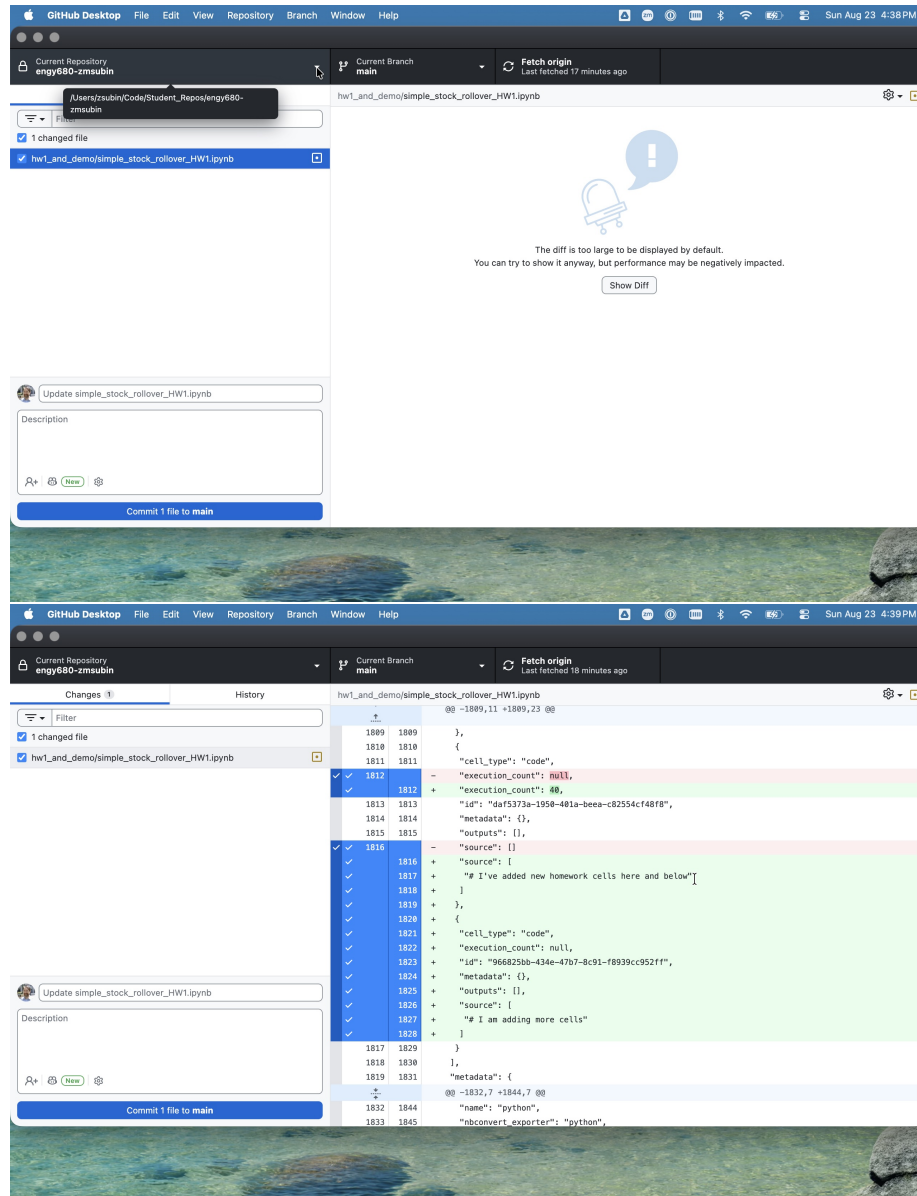
Now save your work back to GitHub using GitHub Desktop.

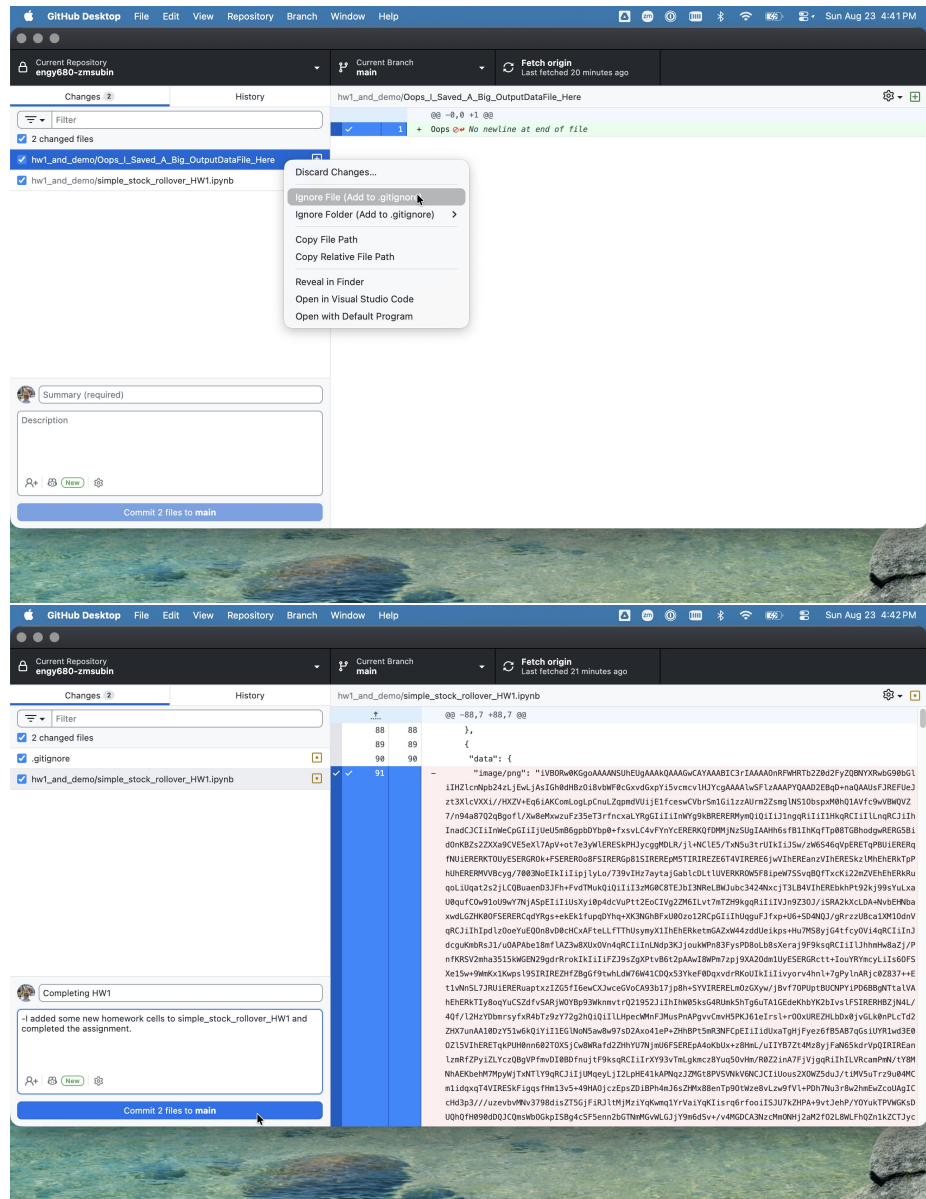
1. Open GitHub Desktop. At the top-left, confirm the current repository dropdown shows [YOUR_USERNAME] / engy680 — not the shared class repo.
2. Click the Changes tab (left side) to see every file you've modified. Click a file to see its diff (added/removed lines) on the right.

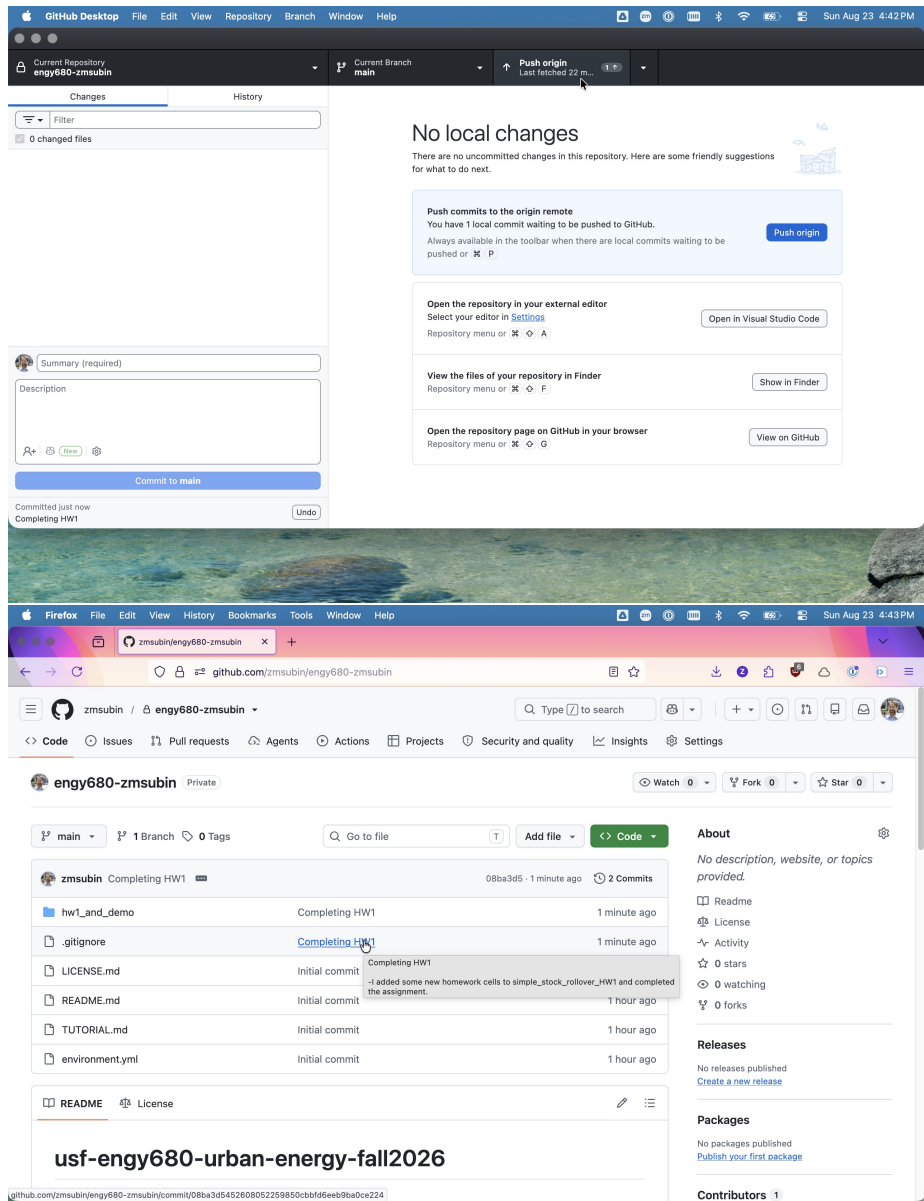
🧐 In contrast to conventional code, (e.g., .py scripts or modules), this diff comparison is not as helpful for Jupyter notebooks which have been saved in a run state because it tracks verbose runtime changes to package versions, binary graphic outputs, etc. Ask about ways to clean this to allow more clean tracking when using notebooks.

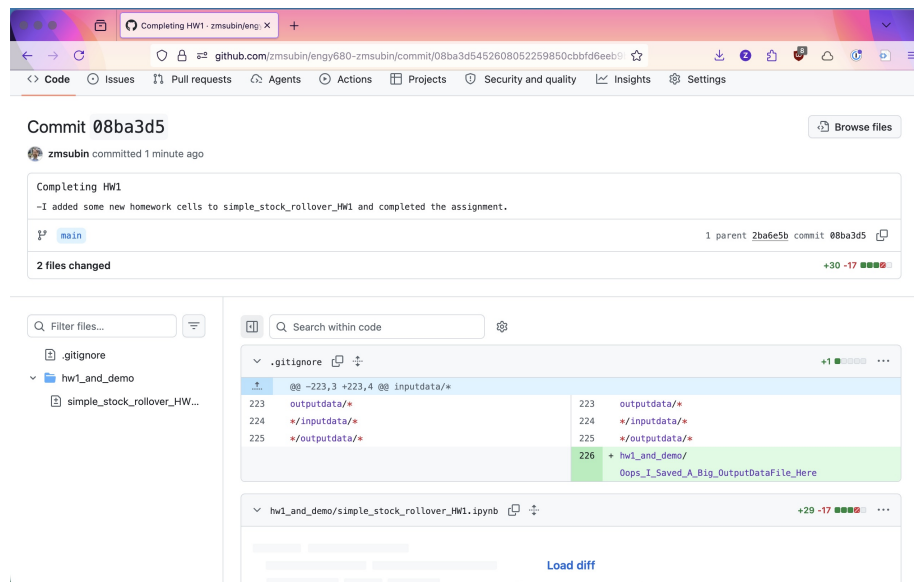
3. Check carefully: only your notebook (.ipynb) and any code files should be checked/staged, along with any appendix chatbot session markdown files or other small addenda. Uncheck (or right-click → Ignore file, which adds it to .gitignore) any large data files, output CSVs, or generated images — these shouldn't go into version control.
4. Write a short Summary (e.g. "Drafting HW1 question 1", "Add HW1 question 1-2 code", "fixed error in question 2 calculation") in the box at bottom left, and optionally a longer Description.

5. Click Commit to main.
6. Click Push origin at the top to upload your commit(s) to GitHub.com.
7. (Optional but reassuring) Open your repo on github.com in a browser and confirm your latest commit appears in the history.









💡 Reminder: You don't need to save copies like HW1_v2.ipynb or HW1_final.ipynb — that's what git/version history is for. Just keep committing to the same file as you make progress.

8. Troubleshooting

Symptom	Likely cause / fix
conda: command not found	Restart your terminal after installing, or (Mac) run <code>conda init zsh</code> (or <code>bash</code>) then restart terminal. On PC, use the Anaconda Prompt, not regular Command Prompt.
conda env create is very slow / hangs on "Solving environment"	Normal — can take 5–15 minutes on first run. If it truly hangs (30+ min), try <code>conda install -n base conda-libmamba-solver</code> then re-run with <code>--solver=libmamba</code> .
GitHub Desktop can't find your repo when cloning	Confirm you accepted the email invite / your GitHub username was sent to the instructor. Try refreshing (sign out/in) in GitHub Desktop.

Symptom	Likely cause / fix
Jupyter opens but can't find installed packages (ModuleNotFoundError)	You likely launched Jupyter from the wrong environment. Confirm the terminal prompt shows (engy680), or in Navigator confirm the environment dropdown is set correctly before clicking Launch.
GitHub Desktop won't let you push (file too large)	GitHub has a ~100 MB per-file hard limit. Un-stage the large file, delete it from tracking, add it to .gitignore, and re-commit.
Mac: "cannot be opened because the developer cannot be verified"	Right-click the app → Open (instead of double-click) the first time, then confirm.